NEXTJS-SKELETON · INTERNAL ENGINEERING HANDBOOK

The AI-Native Engineering Handbook

Why the repository is the harness, how spec-driven development is wired into our skeleton, and how a whole team — PMs, developers, testers, designers — ships through agents without conflicts.

June 2026 · Built and verified with Claude · Companion repo: **NEXTJS-SKELETON**

EXECUTIVE SUMMARY

Code is no longer the constraint. Context and verification are.

For decades, engineering organizations were tooled around one assumption: code is the most expensive thing to produce, so protect it with synchronous human attention. Frontier teams have inverted this. OpenAI's Frontier team shipped roughly **one million lines of code in six months with zero lines typed by a human** — alongside ~250,000 lines of markdown governing agent behavior. The first month was 10× slower than typing the code by hand; by month six, a single 60-hour agent session completed a staff-level refactor with two steering prompts.

Their conclusion, and the premise of our skeleton: **the highest-leverage engineering work is writing things down** — then wiring what was written into machinery the agent cannot ignore. Every piece of feedback given twice is a harness bug.

What we built

NEXTJS-SKELETON is a production-grade Next.js 16 starting point where that philosophy is implemented, not aspirational. It ships with four interlocking systems:

System	What it does	Where it lives
Legibility	Everything an agent needs to do senior-level work is in the repo, pointer-based, current	`AGENTS.md`, `docs/` (30+ files), `.claude/rules/`
Encoded taste	Conventions enforced by gates, not vibes: layer boundaries, typography, docs integrity, strict types	`eslint.config.mjs`, `scripts/check-*`, hooks, `pnpm verify`
Closed verification	Agents prove work against the running app with screenshots, not claims	Playwright CUJs, `artifacts/screenshots/`, `/verify-ui`, `/feature-report`
Async review	Five reviewer personas grade every PR before a human looks	`.claude/agents/`, `docs/personas/`, `claude-review.yml`

Verified, not promised

Everything in the skeleton runs: **lint (boundaries clean) · typecheck · format · docs-link gate (59 files) · typography gate · 9/9 unit tests · production build · 2/2 Playwright CUJs with screenshot evidence**. The demo feature (`task-list`) traveled the entire pipeline — spec → plan → tasks → implementation → verification → report — and the resulting report with real screenshots is committed at `docs/reports/001-task-list.md`.

> **The one-sentence operating model:** Ground → Plan → Implement → Verify → Encode. Humans steer through specs, docs and review gates; agents do the typing; the repo guarantees the floor.

RESEARCH · PART 1

Harness engineering — what frontier teams actually do

Source: Ryan Lopopolo (OpenAI Frontier) on building a 1M-LOC product with agents only; corroborated by Anthropic's Claude Code guidance. Five mechanisms recur, and each has a concrete counterpart in our skeleton:

Frontier practice	Why it works	Our implementation
Repo legible to the agent; `agents.md` points at a docs tree; model decides what to pull	Deterministic steering toward context without blowing the context budget	`AGENTS.md` map + `docs/INDEX.md`; path-scoped digests in `.claude/rules/`
Taste encoded as failing tests (even "petty" ones like curly-quote enforcement)	Agents are trained to make tests pass — a failing gate is prompt injection that cannot be ignored	`check:typography`, `check:docs`, boundaries lint, strict tsconfig
Architecture so modular that "spaghetti is impossible"; ACL-by-folder	Limits the blast radius of any change; enables painted-door experiments and safe parallelism	`app → features → shared → core` enforced by `eslint-plugin-boundaries`; public APIs via `index.ts`
Closed loops: agent drives the app, sees logs/UI, attests end-to-end	"Vibed something up" without proof is the failure mode; evidence builds delegable confidence	Playwright CUJs + `shot()` evidence; `/verify-ui` attestation table
Reviewer personas grade every change before humans; feedback loops back into the repo	Human attention moves to intent and taste; repeated feedback becomes machinery	`.claude/agents/` board + CI persona action; `/encode-lesson` flywheel

> **The economics:** when generation is cheap and parallel, the bottleneck moves to (a) what the agent knows at second zero, and (b) how cheaply you can verify its output. Spend engineering effort exactly there. The corollary — "a billion tokens a day" — only pays off if confidence is the default, which is what the gates buy.

The failure they encoded (and we did too)

Frontier's scheduled-tasks feature collapsed into spaghetti because the harness lacked guardrails; they reverted it and added the missing constraints before retrying. The lesson is institutionalized here as Constitution Article VI and the `/encode-lesson` skill: every correction worth giving twice must end as a doc line, lint rule, test, hook, or persona check — chosen from an explicit escalation ladder, strongest mechanism first.

RESEARCH · PART 2

Spec-driven development — adopted, with its critiques designed in

Sources: GitHub spec-kit (the `/speckit.constitution → specify → clarify → plan → tasks → analyze → implement` workflow) and Birgitta Böckeler's Thoughtworks field evaluation of Kiro, spec-kit and Tessl.

What SDD gets right

- **Specs become the contract** between PM intent and agent implementation — Frontier's PM shipped features from a markdown PRD to a deployed PR within a week, without talking to engineers.
- **A constitution** (immutable principles applied to every change) is the highest-leverage rules file a repo can have.
- **Phase separation** (what → how → steps) catches misunderstandings while they are still cheap words, not code.

Where SDD fails in the field — and our countermeasures

Documented failure mode	Our design response
One heavy workflow for every problem size — "a sledgehammer to crack a nut" (16 acceptance criteria for a small bug)	**Two tracks.** Quick track (no spec) for fixes; spec track only for new behavior or structure. Defined in AGENTS.md, ADR-0003.
Markdown review overload: verbose, repetitive generated files nobody reviews honestly	**One-page templates.** spec/plan/tasks are deliberately lean; the AC→test mapping table is the load-bearing element, not prose volume.
False sense of control: agents ignore parts of large specs	Specs feed **gates**, not just prompts: every AC maps to a test the agent must make pass; `/verify-ui` re-checks ACs against screenshots.
Stale spec piles (spec-anchored ambiguity)	**Specs are anchored to the change, docs to the product.** After ship, `/update-docs` distills the spec into a living feature doc; the spec is archived as history.

> **Net position (ADR-0003):** spec-first always for feature work, spec-anchored during the change's lifetime, never spec-as-source. The long-term source of truth is the docs tree plus the code — kept honest by the docs-link gate and the docs-curator agent.

OPERATING MODEL

One team, four lenses, zero file conflicts

Everyone drives agents; roles differ by what they are accountable for reviewing — not by who may touch the code. When the agent fails someone, engineering fixes the harness, not the person.

Role	Owns	Primary instruments
PO / PM	Problem definition, priorities, acceptance	`/create-spec`, product persona review, `docs/product/`, feature reports as demo artifacts
Developer	Implementation quality and the harness itself	`/plan-feature`, `/implement-feature`, `/new-module`, ADRs, `/encode-lesson`
Tester / QA	Verification strategy and evidence	`/verify-ui`, CUJ registry, `e2e/`, QA persona
Designer	Experience quality, design system	Painted-door experiments, `components/ui`, UX persona, copy + styling conventions

Why parallel work cannot collide

1 · Physical isolation
One feature = one slice = one branch. Features cannot import each other (lint-enforced), so two teams cannot touch the same files.

2 · Declared territory
Every spec lists "areas touched"; `/plan-feature` cross-checks all active specs and flags overlap before any code exists.

3 · Shared layers ride alone
Changes to `components/ui`, `core`, configs or CI ship as dedicated CODEOWNER-reviewed PRs that features rebase onto.

4 · Short-lived branches
Target < 2 days, rebase daily, squash-merge with conventional titles. A long branch is a process smell, not a merge problem.

The feature pipeline (and where evidence appears)

/create-spec → specs/NNN-slug/spec.md ⟶ /plan-feature → plan.md + tasks.md (AC→test mapping) ⟶ /implement-feature (checkpoint commits, gates green) ⟶ /verify-ui → artifacts/screenshots/NNN-slug/ (inspected!)
⟶ /review (5-persona board, P0/P1 fixed) ⟶ /feature-report → docs/reports/NNN-slug.md (+ optional PDF) ⟶ PR (CI + e2e + Claude persona action + human CODEOWNER) ⟶ /update-docs → living feature doc, CUJ registry, spec archived

The PR template mirrors the Definition of Done; branch protection requires the CI and E2E workflows; the persona action posts one consolidated review comment per PR.

AGENT OPTIMIZATION

Tuned for Claude Code & Opus — portable to any agent

Memory without bloat

- **`AGENTS.md` is the single operating model**; `CLAUDE.md` is two paragraphs plus `@AGENTS.md` (the official multi-agent pattern). Codex, Cursor and friends read the same file — no duplication.
- **Path-scoped rules** (`.claude/rules/*.md` with `paths:` frontmatter) load conventions only when matching files are touched — context is spent where it pays.
- **Skills load on demand** (descriptions always visible, bodies only when invoked) — procedures cost nothing until needed.

The nine skills

Skill	Role in the loop
`/create-spec`	Interviews the requester (AskUserQuestion), writes a one-page spec, registers it; refuses to guess on product decisions
`/plan-feature`	Resolves open questions, checks cross-spec conflicts, writes plan + tasks with the AC→test mapping
`/implement-feature`	Executes tasks.md top-down; ticks checkboxes as durable progress (survives compaction); never weakens a gate
`/verify-ui`	Runs CUJ e2e, captures screenshots, *looks at them*, attests per-AC with a verdict table
`/review`	Spawns persona subagents in parallel (context isolation), consolidates P0/P1/P2, drives fixes
`/feature-report`	Diff + screenshots + spec → committed evidence report; `pnpm report:pdf` renders shareable PDF
`/update-docs`	Distills shipped specs into living feature docs; audits doc drift; keeps indexes true
`/new-module`	Scaffolds a slice by mirroring the canonical `task-list` feature — agents cargo-cult, so the canon is kept perfect
`/encode-lesson`	The flywheel: turns any repeated correction into the strongest fitting mechanism (lint → test → hook → persona → rule → doc)

Hard guardrails (hooks — cannot be talked out of)

- **PreToolUse** blocks agent writes to `.env*`, the lockfile, CI workflows, and the constitution — with explanations that teach instead of just refusing.
- **PostToolUse** auto-formats every edited file, so formatting feedback never reaches review or burns agent turns.

Context discipline (Constitution Art. X)

Noisy work goes to subagents (the review board runs in parallel isolation); docs stay under ~200 lines and pointer-based; vendored `llms.txt` references are added per dependency when first

needed; CLI tools are preferred over MCP servers for repeatable integrations (cheaper context, more reliable for agents).

ADOPTION

Rollout roadmap and day-one checklist

Cloning the skeleton for a real project (first hour)

- Fill `docs/product/overview.md` — the first file every agent grounds on.
- Replace `.github/CODEOWNERS` placeholders; protect `main` (require CI + E2E); add the `ANTHROPIC_API_KEY` secret via `/install-github-app` for persona reviews.
- Keep or delete the demo slice (`src/features/task-list` + `src/app/examples`) — if kept, it stays the canonical pattern.
- Ship the first real feature through `/create-spec` end-to-end to validate the loop with your team.

The six-month curve (calibrated to the Frontier experience)

Phase	Focus	Success signal
Months 1–2	Docs discipline: every correction lands in docs/rules/ADRs; team writes specs for real features; expect slower-than-manual at first — that is the investment, not a failure	Agents stop asking questions the repo already answers
Months 3–4	Encode taste: grow `check:*` gates and persona checklists from review findings via `/encode-lesson`; tune CI persona prompts	Human review comments shift from defects to product judgment
Months 5–6	Open the system: PMs ship via specs; designers run painted-door experiments; testers own the CUJ registry	A non-engineer's spec reaches production with zero synchronous engineering time

Health metrics worth tracking

- **Repeat-feedback rate** — the same review note appearing twice means a missing gate (the core harness KPI).
- **Spec → merged lead time** per track, and share of PRs with complete evidence (report + screenshots).
- **Gate catch ratio** — defects caught by machine gates vs. humans (should rise toward the machines).

Sources

R. Lopopolo (OpenAI Frontier) on harness engineering — Product Growth podcast, plus the published roadmap thread · B. Böckeler, "Understanding Spec-Driven Development: Kiro, spec-kit, and Tessl," martinfowler.com (Oct 2025) · github.com/github/spec-kit (workflow, constitution, templates) · Anthropic Claude Code documentation: memory & AGENTS.md import, skills frontmatter, subagents, hooks, claude-code-action v1 · eslint-plugin-boundaries v6 (jsboundaries.dev) · Verified against Next.js 16.2.7 / React 19.2.4 / Tailwind v4 / Zustand 5.0.14 / Motion 12 / Vitest 4 / Playwright 1.60 on 2026-06-07.